

```

import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt
import requests
from io import StringIO
from scipy import stats

import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Conv1D, LSTM, Dense, Dropout, LayerNormalization
from tensorflow.keras.callbacks import EarlyStopping

from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.inspection import permutation_importance

try:
    from xgboost import XGBRegressor
    HAS_XGB = True
except:
    HAS_XGB = False

tf.random.set_seed(42)
np.random.seed(42)

START_DATE = "2014-01-01"
END_DATE = pd.Timestamp.today().strftime("%Y-%m-%d")
LOOKBACK = 60
HORIZON = 5
TRADE_LAG = 1
MAX_TICKERS = None
TRAIN_END = "2020-12-31"
VAL_END = "2022-12-31"
LONG_Q = 0.90
SHORT_Q = 0.10
REBALANCE_EVERY = 5
TRANSACTION_COST = 0.001
MAX_POSITION = 0.08
ROLLING_WINDOW = 26

print("=" * 80)
print("ORIGINAL-FEATURE ALPHA MODEL WITH PROFESSIONAL BACKTESTING EXTENSIONS")
print("=" * 80)

url = "https://en.wikipedia.org/wiki/List_of_S%26P_500_companies"
headers = {"User-Agent": "Mozilla/5.0"}
response = requests.get(url, headers=headers)
response.raise_for_status()
tables = pd.read_html(StringIO(response.text))
sp500_table = tables[0].copy()
sp500_table["ticker"] = sp500_table["Symbol"].str.replace(".", "-", regex=False)
sector_map = sp500_table.set_index("ticker")["GICS Sector"].to_dict()

tickers = sp500_table["ticker"].tolist()
if MAX_TICKERS is not None:

```

```

1. MAX_TICKERS IS NOT NONE.
    tickers = tickers[:MAX_TICKERS]

print(f"Downloading {len(tickers)} tickers from {START_DATE} to {END_DATE}...")

raw = yf.download(
    tickers,
    start=START_DATE,
    end=END_DATE,
    auto_adjust=False,
    group_by="column",
    progress=False,
    threads=True
)

def yahoo_to_long(raw):
    if isinstance(raw.columns, pd.MultiIndex):
        df = raw.stack(level=1).reset_index()
    else:
        df = raw.copy().reset_index()
        df["ticker"] = tickers[0]
    df.columns = [str(c).lower().replace(" ", "_") for c in df.columns]
    if "level_1" in df.columns:
        df = df.rename(columns={"level_1": "ticker"})
    if "price" in df.columns:
        df = df.drop(columns=["price"])
    needed = ["date", "ticker", "open", "high", "low", "close", "adj_close", "volume"]
    df = df[[c for c in needed if c in df.columns]]
    df = df.sort_values(["ticker", "date"]).reset_index(drop=True)
    return df

df = yahoo_to_long(raw)
df["sector"] = df["ticker"].map(sector_map).fillna("Unknown")
df = df.dropna(subset=["adj_close", "close", "volume"])
df = df[(df["adj_close"] > 0) & (df["close"] > 0) & (df["volume"] > 0)]
counts = df.groupby("ticker")["date"].count()
valid_tickers = counts[counts >= 1000].index.tolist()
df = df[df["ticker"].isin(valid_tickers)].copy()
df = df.sort_values(["ticker", "date"]).reset_index(drop=True)

spy = yf.download("SPY", start=START_DATE, end=END_DATE, auto_adjust=False, progress=False)
if isinstance(spy.columns, pd.MultiIndex):
    spy.columns = spy.columns.get_level_values(0)
spy = spy.reset_index()
spy.columns = [str(c).lower().replace(" ", "_") for c in spy.columns]
price_col = "adj_close" if "adj_close" in spy.columns else "close"
spy = spy[["date", price_col]].copy()
spy = spy.rename(columns={price_col: "spy_adj_close"})
spy["spy_ret_1d"] = spy["spy_adj_close"].pct_change(1)
spy["spy_ret_5d"] = spy["spy_adj_close"].pct_change(5)
spy["spy_ret_21d"] = spy["spy_adj_close"].pct_change(21)
spy["spy_ret_63d"] = spy["spy_adj_close"].pct_change(63)
spy["spy_vol_21d"] = spy["spy_ret_1d"].rolling(21).std()

df = df.merge(
    spy[["date", "spy_ret_1d", "spy_ret_5d", "spy_ret_21d", "spy_ret_63d", "spy_vol_21d"]],
    on="date",
    how="left"
)

def add_features(data):
    data = data.sort_values(["ticker", "date"]).copy()

```

```

g = data.groupby("ticker", group_keys=False)

data["ret_1d"] = g["adj_close"].pct_change(1)
data["ret_5d"] = g["adj_close"].pct_change(5)
data["ret_10d"] = g["adj_close"].pct_change(10)
data["ret_21d"] = g["adj_close"].pct_change(21)
data["ret_63d"] = g["adj_close"].pct_change(63)

data["vol_5d"] = g["ret_1d"].rolling(5).std().reset_index(level=0, drop=True)
data["vol_21d"] = g["ret_1d"].rolling(21).std().reset_index(level=0, drop=True)
data["vol_63d"] = g["ret_1d"].rolling(63).std().reset_index(level=0, drop=True)

data["sma_10"] = g["adj_close"].rolling(10).mean().reset_index(level=0, drop=True)
data["sma_21"] = g["adj_close"].rolling(21).mean().reset_index(level=0, drop=True)
data["sma_63"] = g["adj_close"].rolling(63).mean().reset_index(level=0, drop=True)

data["price_to_sma_10"] = data["adj_close"] / data["sma_10"] - 1
data["price_to_sma_21"] = data["adj_close"] / data["sma_21"] - 1
data["price_to_sma_63"] = data["adj_close"] / data["sma_63"] - 1

data["volume_chg_1d"] = g["volume"].pct_change(1)
data["volume_sma_21"] = g["volume"].rolling(21).mean().reset_index(level=0, drop=True)
data["volume_to_sma_21"] = data["volume"] / data["volume_sma_21"] - 1

data["dollar_volume"] = data["adj_close"] * data["volume"]
data["dollar_volume_sma_21"] = g["dollar_volume"].rolling(21).mean().reset_index(level=0, drop=True)
data["dollar_volume_to_sma_21"] = data["dollar_volume"] / data["dollar_volume_sma_21"] - 1

data["daily_range"] = (data["high"] - data["low"]) / data["close"]
data["close_open_ret"] = data["close"] / data["open"] - 1
data["close_position"] = (data["close"] - data["low"]) / (data["high"] - data["low"]) + 0.5

data["excess_ret_1d"] = data["ret_1d"] - data["spy_ret_1d"]
data["excess_ret_5d"] = data["ret_5d"] - data["spy_ret_5d"]

data["beta_63"] = (
    g.apply(lambda x: x["ret_1d"].rolling(63).cov(x["spy_ret_1d"]) / (x["spy_ret_1d"].rolling(63).std()),
            axis=1,
            reset_index=True)
)

entry_price = g["adj_close"].shift(-TRADE_LAG)
exit_price = g["adj_close"].shift(-(TRADE_LAG + HORIZON))
data["target_5d_exec_lagged"] = exit_price / entry_price - 1

return data

print("Engineering original 24-feature set...")
df = add_features(df)

rank_cols = ["ret_5d", "ret_21d", "ret_63d", "vol_21d"]
for col in rank_cols:
    df[col + "_rank"] = df.groupby("date")[col].rank(pct=True)

df["target_5d_cs_z"] = df.groupby("date")["target_5d_exec_lagged"].transform(lambda x: (x - x.mean()) / (x.std() + 1e-8))

cs_features = ["price_to_sma_10", "price_to_sma_21", "price_to_sma_63", "vol_21d", "excess_ret_1d", "excess_ret_5d", "beta_63", "daily_range", "close_position", "close_open_ret"]
for col in cs_features:
    df[f"{col}_cs_z"] = df.groupby("date")[col].transform(lambda x: (x - x.mean()) / (x.std() + 1e-8))

feature_cols = [
    "ret_1d", "ret_5d", "ret_10d", "ret_21d", "ret_63d",
    "vol_5d", "vol_21d", "vol_63d",
    "price_to_sma_10", "price_to_sma_21", "price_to_sma_63",
    "volume_to_sma_21", "dollar_volume_to_sma_21",
    "target_5d_cs_z",
    "price_to_sma_10_cs_z", "price_to_sma_21_cs_z", "price_to_sma_63_cs_z",
    "vol_21d_cs_z", "excess_ret_1d_cs_z", "excess_ret_5d_cs_z",
    "beta_63_cs_z", "daily_range_cs_z", "close_position_cs_z", "close_open_ret_cs_z"
]

```

```

    "price_to_sma_10_cs_z", "price_to_sma_21_cs_z", "price_to_sma_63_cs_z",
    "vol_21d_cs_z", "excess_ret_1d_cs_z",
    "volume_chg_1d", "volume_to_sma_21", "dollar_volume_to_sma_21",
    "daily_range", "close_open_ret", "close_position",
    "excess_ret_5d",
    "ret_5d_rank", "ret_21d_rank", "ret_63d_rank", "vol_21d_rank"
]

df_features_live = df.dropna(subset=feature_cols).copy()
df_model = df.dropna(subset=feature_cols + ["target_5d_exec_lagged", "target_5d_cs_z"]).copy()

df_model.replace([np.inf, -np.inf], np.nan, inplace=True)
df_features_live.replace([np.inf, -np.inf], np.nan, inplace=True)

df_model = df_model.dropna(subset=feature_cols + ["target_5d_exec_lagged", "target_5d_cs_z"])
df_features_live = df_features_live.dropna(subset=feature_cols).sort_values(["ticker", "date"])

train_df = df_model[df_model["date"] <= TRAIN_END].copy()
val_df = df_model[(df_model["date"] > TRAIN_END) & (df_model["date"] <= VAL_END)].copy()
test_df = df_model[df_model["date"] > VAL_END].copy()

scaler = StandardScaler()
scaler.fit(train_df[feature_cols])

for part in [train_df, val_df, test_df, df_features_live]:
    part.loc[:, feature_cols] = scaler.transform(part[feature_cols])

def make_sequences(data, feature_cols, target_col, realized_col, lookback):
    X, y, dates, tickers_out, realized_targets = [], [], [], [], []
    for ticker, sub in data.groupby("ticker"):
        sub = sub.sort_values("date").reset_index(drop=True)
        features = sub[feature_cols].values
        target = sub[target_col].values
        realized = sub[realized_col].values
        sub_dates = sub["date"].values
        for i in range(lookback, len(sub)):
            X.append(features[i - lookback:i])
            y.append(target[i - 1])
            realized_targets.append(realized[i - 1])
            dates.append(sub_dates[i - 1])
            tickers_out.append(ticker)
    return (
        np.array(X, dtype=np.float32),
        np.array(y, dtype=np.float32),
        pd.DataFrame({"date": dates, "ticker": tickers_out, "realized_ret_5d": realized_targets})
    )

def make_live_sequences(data, feature_cols, lookback):
    X, dates, tickers_out = [], [], []
    for ticker, sub in data.groupby("ticker"):
        sub = sub.sort_values("date").reset_index(drop=True)
        if len(sub) < lookback:
            continue
        features = sub[feature_cols].values
        X.append(features[-lookback:])
        dates.append(sub["date"].iloc[-1])
        tickers_out.append(ticker)
    return np.array(X, dtype=np.float32), pd.DataFrame({"date": dates, "ticker": tickers_out})

print("Generating sequences...")
X_train, y_train, meta_train = make_sequences(train_df, feature_cols, "target_5d_cs_z", "target_5d_exec_lagged")
X_val, y_val, meta_val = make_sequences(val_df, feature_cols, "target_5d_cs_z", "target_5d_exec_lagged")
X_test, y_test, meta_test = make_sequences(test_df, feature_cols, "target_5d_cs_z", "target_5d_exec_lagged")

```

```

X_test, y_test, meta_test = make_sequences(test_df, feature_cols, "target_5d_cs_z", "target_5d_cs_z")
X_live, meta_live = make_live_sequences(df_features_live, feature_cols, LOOKBACK)

print(f"Universe after cleaning: {df_model['ticker'].nunique()} stocks")
print(f"Feature count: {len(feature_cols)}")
print(f"Train sequences: {X_train.shape}")
print(f"Validation sequences: {X_val.shape}")
print(f"Test sequences: {X_test.shape}")
print(f"Live/latest scored stocks: {X_live.shape}")

def build_advanced_model(lookback, num_features):
    inputs = Input(shape=(lookback, num_features))
    x = Conv1D(filters=32, kernel_size=3, padding="same", activation="relu")(inputs)
    x = LayerNormalization()(x)
    lstm_out = LSTM(64, return_sequences=True)(x)
    lstm_out = Dropout(0.2)(lstm_out)
    attention_scores = Dense(1, activation="tanh")(lstm_out)
    attention_scores = Flatten()(attention_scores)
    attention_weights = Activation("softmax", name="attention_weights")(attention_scores)
    attention_weights = Reshape((lookback, 1))(attention_weights)
    attended_seq = Multiply()([lstm_out, attention_weights])
    context_vector = Lambda(lambda z: tf.reduce_sum(z, axis=1))(attended_seq)
    dense = Dense(32, activation="swish")(context_vector)
    dense = Dropout(0.1)(dense)
    outputs = Dense(1)(dense)
    model = Model(inputs=inputs, outputs=outputs, name="CNN_LSTM_Attention")
    model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.0005), loss=tf.keras.losses.MeanSquaredError())
    return model

print("Training CNN-LSTM-Attention model...")
model = build_advanced_model(LOOKBACK, len(feature_cols))
early_stop = EarlyStopping(monitor="val_loss", patience=5, min_delta=1e-4, restore_best_weights=True)

history = model.fit(
    X_train,
    y_train,
    validation_data=(X_val, y_val),
    epochs=30,
    batch_size=512,
    callbacks=[early_stop],
    verbose=1
)

pred_test_lstm = model.predict(X_test, batch_size=512).flatten()
pred_live_lstm = model.predict(X_live, batch_size=512).flatten()

X_train_tab = X_train[:, -1, :]
X_test_tab = X_test[:, -1, :]
X_live_tab = X_live[:, -1, :]

ridge = Ridge(alpha=10.0)
ridge.fit(X_train_tab, y_train)
pred_test_ridge = ridge.predict(X_test_tab)
pred_live_ridge = ridge.predict(X_live_tab)

if HAS_XGB:
    tree_model = XGBRegressor(
        n_estimators=400,
        max_depth=3,
        learning_rate=0.03,
        subsample=0.8,

```

```

        colsample_pytree=0.8,
        objective="reg:squarederror",
        random_state=42,
        n_jobs=-1
    )
else:
    tree_model = HistGradientBoostingRegressor(
        max_iter=300,
        learning_rate=0.03,
        max_leaf_nodes=31,
        l2_regularization=0.01,
        random_state=42
    )

print("Training tree baseline...")
tree_model.fit(X_train_tab, y_train)
pred_test_tree = tree_model.predict(X_test_tab)
pred_live_tree = tree_model.predict(X_live_tab)

meta_test_lstm = meta_test.copy()
meta_test_ridge = meta_test.copy()
meta_test_tree = meta_test.copy()

meta_test_lstm["pred_ret_5d"] = pred_test_lstm
meta_test_ridge["pred_ret_5d"] = pred_test_ridge
meta_test_tree["pred_ret_5d"] = pred_test_tree

baseline_tab = test_df[["date", "ticker", "ret_21d", "ret_5d", "target_5d_exec_lagged"]].dropna()
meta_test_mom = baseline_tab[["date", "ticker", "realized_ret_5d"]].copy()
meta_test_mom["pred_ret_5d"] = baseline_tab["ret_21d"]
meta_test_mr = baseline_tab[["date", "ticker", "realized_ret_5d"]].copy()
meta_test_mr["pred_ret_5d"] = -baseline_tab["ret_5d"]

def rank_ic_series(pred_df):
    return pred_df.groupby("date").apply(
        lambda x: x["pred_ret_5d"].corr(x["realized_ret_5d"], method="spearman") if len(x) > 1 else None
    ).dropna()

def ic_summary(pred_df):
    ic = rank_ic_series(pred_df)
    return pd.Series({
        "Mean Rank IC": ic.mean(),
        "IC Std": ic.std(),
        "IC t-stat": ic.mean() / (ic.std() / np.sqrt(len(ic))) if len(ic) > 1 and ic.std() > 0 else None,
        "Positive IC %": (ic > 0).mean(),
        "IC Count": len(ic)
    })

ic_table = pd.DataFrame({
    "CNN-LSTM-Attn": ic_summary(meta_test_lstm),
    "Ridge": ic_summary(meta_test_ridge),
    "Tree/XGBoost": ic_summary(meta_test_tree),
    "Momentum": ic_summary(meta_test_mom),
    "Mean Reversion": ic_summary(meta_test_mr)
})

print("\n" + "=" * 80)
print("RANK IC DIAGNOSTICS")
print("=" * 80)
print(ic_table.round(4))

def capped_normalize(side_df, sign=1, max_position=0.08):

```

```

w = side_df["raw_weight"].values.astype(float)
w = w / (w.sum() + 1e-12)
for _ in range(10):
    w = np.minimum(w, max_position)
    w = w / (w.sum() + 1e-12)
return sign * w

def build_portfolio_history(pred_df, feature_df, long_q=0.90, short_q=0.10, rebalance_every=
enrich_cols = ["date", "ticker", "sector", "vol_21d", "beta_63", "dollar_volume_sma_21"]
available_cols = [c for c in enrich_cols if c in feature_df.columns]
pred_df = pred_df.merge(feature_df[available_cols].drop_duplicates(["date", "ticker"]),

dates = sorted(pred_df["date"].unique())
rebalance_dates = dates[::rebalance_every]
portfolio_rows = []
holding_rows = []
prev_weights = pd.Series(dtype=float)

for d in rebalance_dates:
    day_data = pred_df[pred_df["date"] == d].copy()
    if len(day_data) < 20:
        continue

    long_cut = day_data["pred_ret_5d"].quantile(long_q)
    short_cut = day_data["pred_ret_5d"].quantile(short_q)

    longs = day_data[day_data["pred_ret_5d"] >= long_cut].copy()
    shorts = day_data[day_data["pred_ret_5d"] <= short_cut].copy()

    if len(longs) == 0 or len(shorts) == 0:
        continue

    longs["side"] = "Long"
    shorts["side"] = "Short"

    longs["raw_weight"] = 1.0 / (longs["vol_21d"].abs() + 1e-6)
    shorts["raw_weight"] = 1.0 / (shorts["vol_21d"].abs() + 1e-6)

    longs["weight"] = capped_normalize(longs, sign=1, max_position=max_position)
    shorts["weight"] = capped_normalize(shorts, sign=-1, max_position=max_position)

    positions = pd.concat([longs, shorts], axis=0).copy()
    target_weights = positions.set_index("ticker")["weight"]

    turnover = target_weights.sub(prev_weights, fill_value=0).abs().sum()
    tc = cost * turnover

    positions["pnl_contribution"] = positions["weight"] * positions["realized_ret_5d"]

    long_return = positions.loc[positions["weight"] > 0, "pnl_contribution"].sum()
    short_return = positions.loc[positions["weight"] < 0, "pnl_contribution"].sum()
    gross_return = positions["pnl_contribution"].sum()
    net_return = gross_return - tc

    gross_exposure = positions["weight"].abs().sum()
    net_exposure = positions["weight"].sum()
    long_exposure = positions.loc[positions["weight"] > 0, "weight"].sum()
    short_exposure = positions.loc[positions["weight"] < 0, "weight"].sum()

    net_beta = (positions["weight"] * positions["beta_63"].fillna(0)).sum()
    long_beta = (positions.loc[positions["weight"] > 0, "weight"] * positions.loc[positi
    short_beta = (positions.loc[positions["weight"] < 0, "weight"] * positions.loc[posit

```

```

concentration_hhi = (positions["weight"].abs() ** 2).sum()
effective_positions = 1 / concentration_hhi if concentration_hhi > 0 else np.nan

portfolio_rows.append({
    "date": d,
    "gross_return": gross_return,
    "net_return": net_return,
    "long_return": long_return,
    "short_return": short_return,
    "transaction_cost": tc,
    "turnover": turnover,
    "gross_exposure": gross_exposure,
    "net_exposure": net_exposure,
    "long_exposure": long_exposure,
    "short_exposure": short_exposure,
    "net_beta": net_beta,
    "long_beta": long_beta,
    "short_beta": short_beta,
    "n_long": (positions["side"] == "Long").sum(),
    "n_short": (positions["side"] == "Short").sum(),
    "concentration_hhi": concentration_hhi,
    "effective_positions": effective_positions,
    "largest_long": positions["weight"].max(),
    "largest_short": positions["weight"].min(),
    "avg_pred_long": positions.loc[positions["side"] == "Long", "pred_ret_5d"].mean(),
    "avg_pred_short": positions.loc[positions["side"] == "Short", "pred_ret_5d"].mean(),
    "avg_realized_long": positions.loc[positions["side"] == "Long", "realized_ret_5d"].mean(),
    "avg_realized_short": positions.loc[positions["side"] == "Short", "realized_ret_5d"].mean()
})

for _, row in positions.iterrows():
    holding_rows.append({
        "date": d,
        "ticker": row["ticker"],
        "sector": row.get("sector", "Unknown"),
        "side": row["side"],
        "weight": row["weight"],
        "pred_score": row["pred_ret_5d"],
        "realized_ret_5d": row["realized_ret_5d"],
        "pnl_contribution": row["pnl_contribution"],
        "vol_21d": row.get("vol_21d", np.nan),
        "beta_63": row.get("beta_63", np.nan),
        "dollar_volume_sma_21": row.get("dollar_volume_sma_21", np.nan)
    })

prev_weights = target_weights

portfolio_history = pd.DataFrame(portfolio_rows)
holdings_history = pd.DataFrame(holding_rows)

if len(portfolio_history) > 0:
    portfolio_history["cum_return"] = (1 + portfolio_history["net_return"]).cumprod()
    portfolio_history["cum_long"] = (1 + portfolio_history["long_return"]).cumprod()
    portfolio_history["cum_short"] = (1 + portfolio_history["short_return"]).cumprod()
    portfolio_history["running_max"] = portfolio_history["cum_return"].cummax()
    portfolio_history["drawdown"] = portfolio_history["cum_return"] / portfolio_history["running_max"]
    portfolio_history["rolling_return"] = portfolio_history["net_return"].rolling(ROLLING_WINDOW).mean()
    portfolio_history["rolling_vol"] = portfolio_history["net_return"].rolling(ROLLING_WINDOW).std()
    portfolio_history["rolling_sharpe"] = (
        portfolio_history["net_return"].rolling(ROLLING_WINDOW).mean() * (252 / rebalancing_frequency) /
        portfolio_history["rolling_vol"]
    )

```

```

    / / portfolio_history[rolling_vol]

return portfolio_history, holdings_history

def performance_summary(bt, return_col="net_return", periods_per_year=252 / REBALANCE_EVER
    if len(bt) == 0:
        return pd.Series(dtype=float)
    r = bt[return_col].dropna()
    ann_return = (1 + r).prod() ** (periods_per_year / len(r)) - 1
    ann_vol = r.std() * np.sqrt(periods_per_year)
    sharpe = ann_return / ann_vol if ann_vol != 0 else np.nan
    cumulative = (1 + r).cumprod()
    running_max = cumulative.cummax()
    drawdown = cumulative / running_max - 1
    t_stat = r.mean() / (r.std() / np.sqrt(len(r))) if len(r) > 1 and r.std() > 0 else np.r
    return pd.Series({
        "Annualized Return": ann_return,
        "Annualized Volatility": ann_vol,
        "Sharpe Ratio": sharpe,
        "Return t-stat": t_stat,
        "Max Drawdown": drawdown.min(),
        "Hit Rate": (r > 0).mean(),
        "Average Turnover": bt["turnover"].mean() if "turnover" in bt.columns else np.nan,
        "Average Net Beta": bt["net_beta"].mean() if "net_beta" in bt.columns else np.nan,
        "Average Gross Exposure": bt["gross_exposure"].mean() if "gross_exposure" in bt.co
        "Average Effective Positions": bt["effective_positions"].mean() if "effective_posit
    })

print("Running portfolio backtests...")
bt_lstm, holdings_lstm = build_portfolio_history(meta_test_lstm, test_df, LONG_Q, SHORT_Q,
bt_ridge, holdings_ridge = build_portfolio_history(meta_test_ridge, test_df, LONG_Q, SHORT
bt_tree, holdings_tree = build_portfolio_history(meta_test_tree, test_df, LONG_Q, SHORT_Q,
bt_mom, holdings_mom = build_portfolio_history(meta_test_mom, test_df, LONG_Q, SHORT_Q, RE
bt_mr, holdings_mr = build_portfolio_history(meta_test_mr, test_df, LONG_Q, SHORT_Q, REBAL

comparison = pd.DataFrame({
    "CNN-LSTM-Attn": performance_summary(bt_lstm),
    "Ridge": performance_summary(bt_ridge),
    "Tree/XGBoost": performance_summary(bt_tree),
    "Momentum": performance_summary(bt_mom),
    "Mean Reversion": performance_summary(bt_mr)
})

print("\n" + "=" * 80)
print("BACKTEST PERFORMANCE SUMMARY")
print("=" * 80)
print(comparison.round(4))

def yearly_performance(bt):
    temp = bt.copy()
    temp["year"] = temp["date"].dt.year
    rows = []
    for year, sub in temp.groupby("year"):
        s = performance_summary(sub)
        s["Year"] = year
        rows.append(s)
    return pd.DataFrame(rows).set_index("Year")

print("\n" + "=" * 80)
print("YEARLY PERFORMANCE: CNN-LSTM-ATTENTION")
print("=" * 80)
yearly_lstm = yearly_performance(bt_lstm)

```

```

print(yearly_lstm.round(4))

ic_lstm = rank_ic_series(meta_test_lstm).rename("rank_ic").reset_index()
ic_lstm["rolling_ic"] = ic_lstm["rank_ic"].rolling(60).mean()
ic_lstm["rolling_ic_tstat"] = ic_lstm["rank_ic"].rolling(60).apply(lambda x: x.mean() / (x.

print("\n" + "=" * 80)
print("ROLLING IC SUMMARY")
print("=" * 80)
print(ic_lstm[["date", "rank_ic", "rolling_ic", "rolling_ic_tstat"]].tail(10).round(4))

sector_exposure = holdings_lstm.groupby(["date", "sector"])["weight"].sum().reset_index()
avg_sector_exposure = sector_exposure.groupby("sector")["weight"].agg(["mean", "std", "min"]
sector_pnl = holdings_lstm.groupby("sector")["pnl_contribution"].sum().sort_values(ascending=
ticker_pnl = holdings_lstm.groupby("ticker")["pnl_contribution"].sum().sort_values(ascending=

print("\n" + "=" * 80)
print("AVERAGE SECTOR EXPOSURE: CNN-LSTM-ATTENTION")
print("=" * 80)
print(avg_sector_exposure.round(4))

print("\n" + "=" * 80)
print("SECTOR PNL CONTRIBUTION: CNN-LSTM-ATTENTION")
print("=" * 80)
print(sector_pnl.round(4))

print("\n" + "=" * 80)
print("TOP 15 TICKER PNL CONTRIBUTORS")
print("=" * 80)
print(ticker_pnl.head(15).round(4))

print("\n" + "=" * 80)
print("BOTTOM 15 TICKER PNL CONTRIBUTORS")
print("=" * 80)
print(ticker_pnl.tail(15).round(4))

holding_stability = holdings_lstm.groupby("ticker").agg(
    times_held=("date", "count"),
    avg_weight=("weight", "mean"),
    avg_abs_weight=("weight", lambda x: x.abs().mean()),
    cumulative_pnl=("pnl_contribution", "sum"),
    avg_pred_score=("pred_score", "mean"),
    avg_realized_ret=("realized_ret_5d", "mean")
).sort_values("times_held", ascending=False)

print("\n" + "=" * 80)
print("HOLDING STABILITY")
print("=" * 80)
print(holding_stability.head(25).round(4))

meta_live_lstm = meta_live.copy()
meta_live_lstm["pred_ret_5d"] = pred_live_lstm

latest_feature_df = df_features_live.copy()
latest_live = meta_live_lstm.merge(
    latest_feature_df[["date", "ticker", "sector", "vol_21d", "beta_63", "dollar_volume_sm
on=["date", "ticker"],
    how="left"
)

latest_date = latest_live["date"].max()
latest_live = latest_live[latest_live["date"] == latest_date].copy()

```

```

long_cut = latest_live["pred_ret_5d"].quantile(LONG_Q)
short_cut = latest_live["pred_ret_5d"].quantile(SHORT_Q)

latest longs = latest_live[latest_live["pred_ret_5d"] >= long_cut].copy()
latest_shorts = latest_live[latest_live["pred_ret_5d"] <= short_cut].copy()

latest longs["side"] = "Long"
latest_shorts["side"] = "Short"

latest longs["raw_weight"] = 1.0 / (latest longs["vol_21d"].abs() + 1e-6)
latest_shorts["raw_weight"] = 1.0 / (latest_shorts["vol_21d"].abs() + 1e-6)

latest longs["weight"] = capped_normalize(latest longs, 1, MAX_POSITION)
latest_shorts["weight"] = capped_normalize(latest_shorts, -1, MAX_POSITION)

latest_portfolio = pd.concat([latest longs, latest_shorts], axis=0).sort_values("weight", ascending=False)

print("\n" + "=" * 80)
print(f"LATEST LIVE-STYLE MODEL PORTFOLIO SIGNAL: {pd.to_datetime(latest_date).strftime('%Y-%m-%d %H:%M:%S')}")
print("=" * 80)
print(latest_portfolio[["ticker", "sector", "side", "pred_ret_5d", "weight", "vol_21d", "beta"]])

print("\n" + "=" * 80)
print("FEATURE IMPORTANCE: RIDGE AND TREE MODEL")
print("=" * 80)

ridge_importance = pd.Series(np.abs(ridge.coef_), index=feature_cols).sort_values(ascending=False)
print("\nTop Ridge Features:")
print(ridge_importance.head(24).round(4))

sample_n = min(5000, len(X_test_tab))
sample_idx = np.random.choice(len(X_test_tab), size=sample_n, replace=False)

try:
    perm = permutation_importance(
        tree_model,
        X_test_tab[sample_idx],
        y_test[sample_idx],
        n_repeats=3,
        random_state=42,
        n_jobs=-1
    )
    perm_importance = pd.Series(perm.importances_mean, index=feature_cols).sort_values(ascending=False)
    print("\nTop Tree Permutation Features:")
    print(perm_importance.head(24).round(6))
except Exception as e:
    print("\nPermutation importance skipped:", e)

plt.figure(figsize=(14, 6))
plt.plot(bt_lstm["date"], bt_lstm["cum_return"], label="CNN-LSTM-Attn")
plt.plot(bt_ridge["date"], bt_ridge["cum_return"], label="Ridge", alpha=0.8)
plt.plot(bt_tree["date"], bt_tree["cum_return"], label="Tree/XGBoost", alpha=0.8)
plt.plot(bt_mom["date"], bt_mom["cum_return"], label="Momentum", alpha=0.7)
plt.plot(bt_mr["date"], bt_mr["cum_return"], label="Mean Reversion", alpha=0.7)
plt.title("Out-of-Sample Portfolio Evolution")
plt.xlabel("Date")
plt.ylabel("Cumulative Growth of $1")
plt.legend()
plt.grid(True)
plt.show()

```

```

plt.figure(figsize=(14, 5))
plt.plot(bt_lstm["date"], bt_lstm["cum_return"], label="Total Net Portfolio")
plt.plot(bt_lstm["date"], bt_lstm["cum_long"], label="Long Book")
plt.plot(bt_lstm["date"], bt_lstm["cum_short"], label="Short Book")
plt.title("CNN-LSTM Portfolio Evolution: Total, Long Book, Short Book")
plt.xlabel("Date")
plt.ylabel("Cumulative Growth of $1")
plt.legend()
plt.grid(True)
plt.show()

plt.figure(figsize=(14, 5))
plt.plot(bt_lstm["date"], bt_lstm["drawdown"])
plt.title("CNN-LSTM Drawdown Over Time")
plt.xlabel("Date")
plt.ylabel("Drawdown")
plt.grid(True)
plt.show()

plt.figure(figsize=(14, 5))
plt.plot(bt_lstm["date"], bt_lstm["rolling_sharpe"])
plt.axhline(0, linestyle="--", linewidth=1)
plt.title("CNN-LSTM Rolling 6-Month Sharpe")
plt.xlabel("Date")
plt.ylabel("Rolling Sharpe")
plt.grid(True)
plt.show()

plt.figure(figsize=(14, 5))
plt.plot(ic_lstm["date"], ic_lstm["rank_ic"], alpha=0.4, label="Daily Rank IC")
plt.plot(ic_lstm["date"], ic_lstm["rolling_ic"], label="Rolling 60-Day Rank IC")
plt.axhline(0, linestyle="--", linewidth=1)
plt.title("CNN-LSTM Rank IC Over Time")
plt.xlabel("Date")
plt.ylabel("Rank IC")
plt.legend()
plt.grid(True)
plt.show()

plt.figure(figsize=(14, 5))
plt.plot(bt_lstm["date"], bt_lstm["turnover"], label="Turnover")
plt.plot(bt_lstm["date"], bt_lstm["transaction_cost"], label="Transaction Cost")
plt.title("Turnover and Transaction Costs")
plt.xlabel("Date")
plt.ylabel("Value")
plt.legend()
plt.grid(True)
plt.show()

plt.figure(figsize=(14, 5))
plt.plot(bt_lstm["date"], bt_lstm["net_beta"], label="Net Beta")
plt.plot(bt_lstm["date"], bt_lstm["gross_exposure"], label="Gross Exposure")
plt.plot(bt_lstm["date"], bt_lstm["net_exposure"], label="Net Dollar Exposure")
plt.axhline(0, linestyle="--", linewidth=1)
plt.title("Portfolio Exposure and Beta Over Time")
plt.xlabel("Date")
plt.ylabel("Exposure")
plt.legend()
plt.grid(True)
plt.show()

plt.figure(figsize=(14, 5))

```

```

plt.plot(bt_lstm["date"], bt_lstm["concentration_hhi"], label="HHI Concentration")
plt.plot(bt_lstm["date"], bt_lstm["effective_positions"], label="Effective Number of Posit:
plt.title("Portfolio Concentration Over Time")
plt.xlabel("Date")
plt.ylabel("Value")
plt.legend()
plt.grid(True)
plt.show()

monthly = bt_lstm.copy()
monthly["month"] = monthly["date"].dt.to_period("M").astype(str)
monthly_returns = monthly.groupby("month")["net_return"].apply(lambda x: (1 + x).prod() - 1)

plt.figure(figsize=(14, 5))
monthly_returns.plot(kind="bar")
plt.title("Monthly Strategy Returns: CNN-LSTM")
plt.xlabel("Month")
plt.ylabel("Monthly Return")
plt.grid(axis="y")
plt.tight_layout()
plt.show()

plt.figure(figsize=(14, 5))
ticker_pnl.head(15).sort_values().plot(kind="barh")
plt.title("Top 15 Ticker PnL Contributors")
plt.xlabel("Cumulative PnL Contribution")
plt.grid(axis="x")
plt.tight_layout()
plt.show()

plt.figure(figsize=(14, 5))
sector_pnl.sort_values().plot(kind="barh")
plt.title("Sector PnL Contribution")
plt.xlabel("Cumulative PnL Contribution")
plt.grid(axis="x")
plt.tight_layout()
plt.show()

print("\n" + "=" * 80)
print("FINAL INTERPRETATION")
print("=" * 80)
print("This version keeps the original 24 model input features but upgrades the research p:

```


ORIGINAL-FEATURE ALPHA MODEL WITH PROFESSIONAL BACKTESTING EXTENSIONS

Downloading 503 tickers from 2014-01-01 to 2026-04-29...

Engineering original 24-feature set...

Generating sequences...

Universe after cleaning: 496 stocks

Feature count: 24

Train sequences: (773072, 60, 24)

Validation sequences: (218521, 60, 24)

Test sequences: (379737, 60, 24)

Live/latest scored stocks: (496, 60, 24)

Training CNN-LSTM-Attention model...

Epoch 1/30

1510/1510 ————— **23s** 12ms/step - loss: 0.3562 - mae: 0.6929 - val_loss: 0.3

Epoch 2/30

1510/1510 ————— **15s** 10ms/step - loss: 0.3545 - mae: 0.6907 - val_loss: 0.3

Epoch 3/30

1510/1510 ————— **15s** 10ms/step - loss: 0.3520 - mae: 0.6876 - val_loss: 0.3

Epoch 4/30

1510/1510 ————— **15s** 10ms/step - loss: 0.3486 - mae: 0.6832 - val_loss: 0.3

Epoch 5/30

1510/1510 ————— **15s** 10ms/step - loss: 0.3451 - mae: 0.6788 - val_loss: 0.3

Epoch 6/30

1510/1510 ————— **15s** 10ms/step - loss: 0.3417 - mae: 0.6745 - val_loss: 0.4

Epoch 6: early stopping

Restoring model weights from the end of the best epoch: 1.

742/742 ————— **3s** 3ms/step

1/1 ————— **0s** 69ms/step

Training tree baseline...

RANK IC DIAGNOSTICS

	CNN-LSTM-Attn	Ridge	Tree/XGBoost	Momentum	Mean Reversion
Mean Rank IC	0.0306	0.0111	0.0226	-0.0005	0.0009
IC Std	0.1794	0.1812	0.1529	0.1792	0.1684
IC t-stat	4.7262	1.6936	4.0874	-0.0862	0.1528
Positive IC %	0.5574	0.5131	0.5431	0.5157	0.4746
IC Count	766.0000	766.0000	766.0000	826.0000	826.0000

Running portfolio backtests...

BACKTEST PERFORMANCE SUMMARY

	CNN-LSTM-Attn	Ridge	Tree/XGBoost	Momentum	\
Annualized Return	0.0499	0.0049	0.0385	-0.1060	
Annualized Volatility	0.1962	0.1829	0.1780	0.1940	
Sharpe Ratio	0.2544	0.0270	0.2164	-0.5466	
Return t-stat	0.6023	0.2068	0.5245	-0.8716	
Max Drawdown	-0.2221	-0.2778	-0.2064	-0.3955	
Hit Rate	0.5195	0.5584	0.4870	0.4759	
Average Turnover	1.7983	3.2211	2.5701	2.4223	
Average Net Beta	0.4561	0.5281	0.5633	0.0432	
Average Gross Exposure	2.0000	2.0000	2.0000	2.0000	
Average Effective Positions	11.8886	11.9358	11.7729	11.5273	

Mean Reversion

Annualized Return	-0.1463
Annualized Volatility	0.1912
Sharpe Ratio	-0.7654
Return t-stat	-1.3245
Max Drawdown	-0.4249
Hit Rate	0.4277
Average Turnover	3.7169
Average Net Beta	-0.0222
Average Gross Exposure	2.0000

Average Effective Positions

11.6126

=====

YEARLY PERFORMANCE: CNN-LSTM-ATTENTION

=====

	Annualized Return	Annualized Volatility	Sharpe Ratio	Return t-stat \
Year				
2023.0	-0.0601	0.1232	-0.4879	-0.3894
2024.0	0.0688	0.1619	0.4249	0.4892
2025.0	0.1723	0.2577	0.6687	0.7380
2026.0	-0.0863	0.2308	-0.3741	-0.1546

	Max Drawdown	Hit Rate	Average Turnover	Average Net Beta \
Year				
2023.0	-0.1408	0.5385	1.8457	0.2016
2024.0	-0.1492	0.5400	1.7596	0.3532
2025.0	-0.1850	0.5000	1.8586	0.5840
2026.0	-0.1264	0.4667	1.6030	1.0346

	Average Gross Exposure	Average Effective Positions
Year		
2023.0	2.0	11.7146
2024.0	2.0	11.8192
2025.0	2.0	11.8388
2026.0	2.0	12.7387

=====

ROLLING IC SUMMARY

=====

	date	rank_ic	rolling_ic	rolling_ic_tstat
756	2026-04-06	0.3065	-0.0112	-0.5301
757	2026-04-07	0.3985	-0.0042	-0.1906
758	2026-04-08	0.4752	0.0035	0.1469
759	2026-04-09	0.5434	0.0130	0.5121
760	2026-04-10	0.4276	0.0195	0.7439
761	2026-04-13	0.3729	0.0252	0.9361
762	2026-04-14	0.3052	0.0291	1.0674
763	2026-04-15	-0.1149	0.0260	0.9497
764	2026-04-16	-0.0603	0.0237	0.8656
765	2026-04-17	-0.1506	0.0226	0.8223

=====

AVERAGE SECTOR EXPOSURE: CNN-LSTM-ATTENTION

=====

	mean	std	min	max
sector				
Information Technology	0.1633	0.1639	-0.3514	0.5848
Consumer Discretionary	0.0216	0.1034	-0.2733	0.2520
Communication Services	0.0160	0.0739	-0.2150	0.2277
Industrials	0.0043	0.1154	-0.2652	0.2907
Financials	-0.0095	0.1327	-0.3980	0.2831
Health Care	-0.0155	0.1221	-0.2948	0.2671
Materials	-0.0164	0.0822	-0.3021	0.1378
Energy	-0.0325	0.1346	-0.3151	0.3563
Consumer Staples	-0.0327	0.1031	-0.3178	0.2200
Real Estate	-0.0449	0.0872	-0.2693	0.2295
Utilities	-0.0560	0.1019	-0.3908	0.1180

=====

SECTOR PNL CONTRIBUTION: CNN-LSTM-ATTENTION

=====

sector	
Information Technology	0.3509
Industrials	0.1149
Consumer Discretionary	0.1026
Materials	0.0393
Consumer Staples	0.0094
Utilities	0.0081

```

Communication Services -0.0069
Real Estate -0.0161
Health Care -0.0245
Financials -0.0424
Energy -0.0517
Name: pnl_contribution, dtype: float64

```

TOP 15 TICKER PNL CONTRIBUTORS

```

ticker
FIX      0.0302
DVA      0.0254
CRL      0.0240
ELV      0.0237
LITE     0.0234
AVGO     0.0230
MPWR     0.0229
PLTR     0.0224
ANET     0.0221
MOS      0.0214
HUM      0.0212
CVNA     0.0205
INTC     0.0184
PANW     0.0182
FSLR     0.0180
Name: pnl_contribution, dtype: float64

```

BOTTOM 15 TICKER PNL CONTRIBUTORS

```

ticker
WST      -0.0140
TRMB     -0.0150
CAH      -0.0176
APO      -0.0185
ALGN     -0.0187
CTRA     -0.0188
LULU     -0.0189
ISRG     -0.0207
PYPL     -0.0217
TPL      -0.0218
DLTR     -0.0236
MRNA     -0.0241
DG       -0.0272
FISV     -0.0373
CHTR     -0.0454
Name: pnl_contribution, dtype: float64

```

HOLDING STABILITY

```

            times_held  avg_weight  avg_abs_weight  cumulative_pnl  \
ticker
CVNA           140      0.0061      0.0061      0.0205
SMCI           137      0.0038      0.0038      0.0049
COIN           136      0.0046      0.0046      0.0099
PLTR           133      0.0069      0.0069      0.0224
APP            127      0.0059      0.0061      0.0147
COHR           119      0.0049      0.0062      0.0123
FSLR           113      0.0122      0.0125      0.0180
ALB            109      0.0077      0.0084      0.0005
MRNA           108      0.0086      0.0087     -0.0241
TSLA           106      0.0103      0.0103      0.0069
HOOD           106      0.0068      0.0068      0.0114
VRT            103      0.0071      0.0073      0.0167
SATS            97      0.0075      0.0113      0.0019

```

LITE	95	0.0047	0.0071	0.0234
TPL	95	0.0138	0.0243	-0.0218
AMD	91	0.0102	0.0102	0.0023
DDOG	89	0.0103	0.0204	0.0067
FIX	88	0.0156	0.0156	0.0302
NVDA	87	0.0154	0.0154	0.0073
PSKY	84	0.0113	0.0132	0.0089
TTD	80	0.0110	0.0138	0.0130
WBD	80	0.0123	0.0123	0.0119
TER	79	0.0023	0.0090	0.0175
ON	79	0.0099	0.0113	-0.0073
BLDR	79	0.0167	0.0180	0.0027

	avg_pred_score	avg_realized_ret
ticker		
CVNA	0.1475	0.0370
SMCI	0.1609	0.0203
COIN	0.1545	0.0168
PLTR	0.1330	0.0268
APP	0.1378	0.0215
COHR	0.1216	0.0107
FSLR	0.1385	0.0045
ALB	0.1202	0.0030
MRNA	0.1579	-0.0024
TSLA	0.1102	0.0160
HOOD	0.1435	0.0187
VRT	0.1447	0.0158
SATS	0.1252	0.0272
LITE	0.1137	0.0266
TPL	0.0697	0.0039
AMD	0.1104	0.0066
DDOG	0.0872	0.0134
FIX	0.1145	0.0211
NVDA	0.1204	0.0117
PSKY	0.1218	-0.0052
TTD	0.1106	0.0106
WBD	0.1144	0.0068
TER	0.0617	0.0167
ON	0.1031	-0.0011
BLDR	0.0982	0.0024

=====

LATEST LIVE-STYLE MODEL PORTFOLIO SIGNAL: 2026-04-28

=====

	ticker	sector	side	pred_ret_5d	weight	vol_21d \
10	ADP	Industrials	Long	0.1334	0.0800	-0.0595
274	LDOS	Industrials	Long	0.1854	0.0800	-0.1253
448	TYL	Information Technology	Long	0.1185	0.0800	0.1438
378	PTC	Information Technology	Long	0.1192	0.0800	0.0389
431	TKO	Communication Services	Long	0.1037	0.0662	0.1805
..	...	...	...	...	...	...
225	HLT	Consumer Discretionary	Short	-0.1297	-0.0520	0.1005
417	SWKS	Information Technology	Short	-0.1029	-0.0594	0.0879
138	DGX	Health Care	Short	-0.0956	-0.0667	-0.0783
336	NUE	Materials	Short	-0.1064	-0.0672	0.0777
375	PSA	Real Estate	Short	-0.1200	-0.0800	-0.0331

	beta_63	dollar_volume_sma_21
10	0.2351	5.890770e+08
274	0.2256	1.271891e+08
448	0.6550	1.501380e+08
378	0.6880	1.501632e+08
431	0.8677	3.046936e+08
..	...	...
225	1.0178	4.998687e+08
417	0.7739	1.684511e+08
138	-0.0172	2.043246e+08
336	0.9666	3.015631e+08

```

375      0.6810      3.187505e+08

```

[100 rows x 8 columns]

===== FEATURE IMPORTANCE: RIDGE AND TREE MODEL =====

Top Ridge Features:

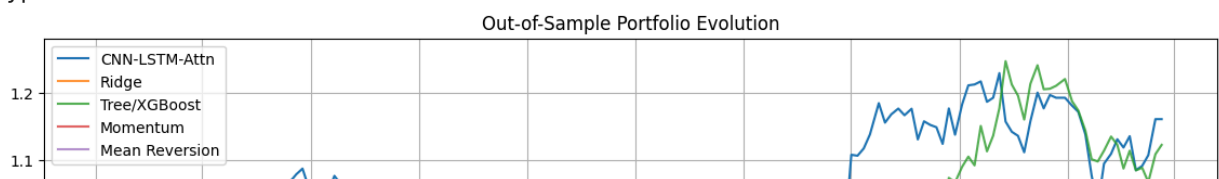
excess_ret_5d	0.0619
vol_21d_cs_z	0.0369
vol_21d	0.0277
vol_63d	0.0264
ret_5d_rank	0.0209
ret_63d_rank	0.0190
price_to_sma_21_cs_z	0.0181
vol_21d_rank	0.0179
ret_21d_rank	0.0173
ret_5d	0.0163
ret_1d	0.0117
price_to_sma_63_cs_z	0.0103
daily_range	0.0087
excess_ret_1d_cs_z	0.0086
vol_5d	0.0079
price_to_sma_10_cs_z	0.0069
dollar_volume_to_sma_21	0.0069
ret_63d	0.0063
ret_21d	0.0031
ret_10d	0.0018
volume_chg_1d	0.0013
close_position	0.0010
volume_to_sma_21	0.0008
close_open_ret	0.0006

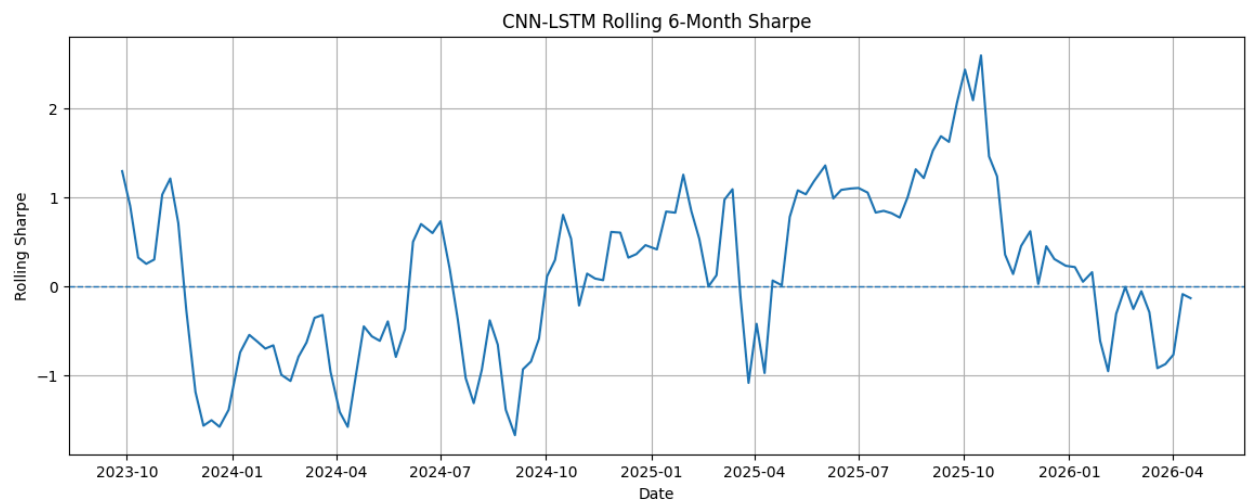
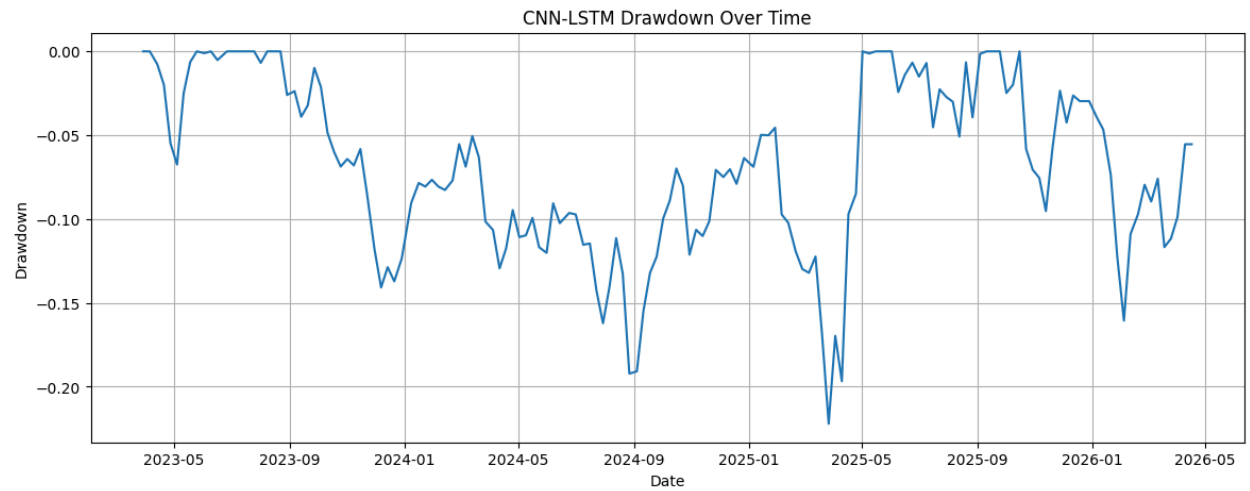
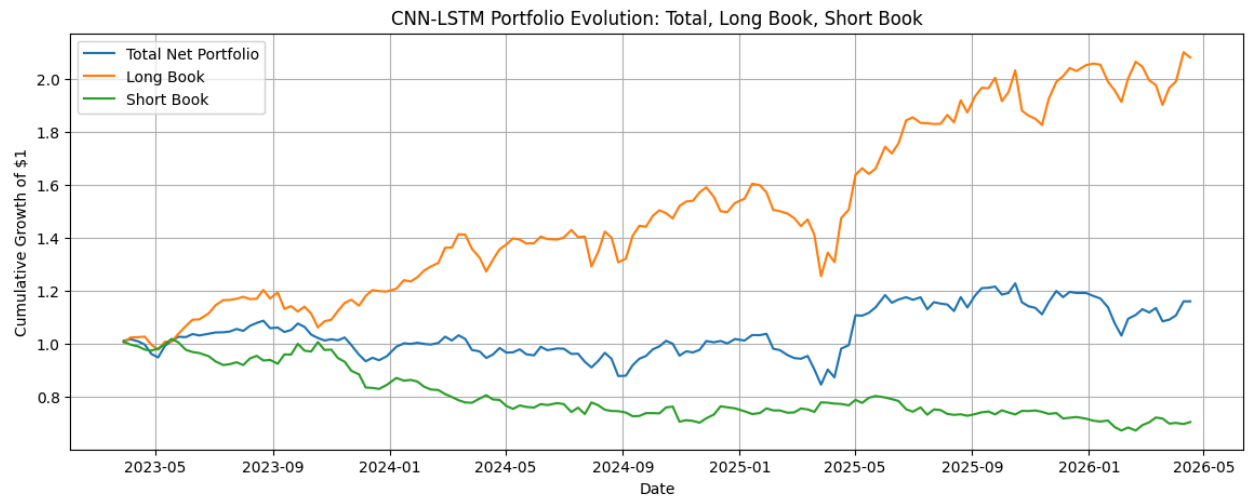
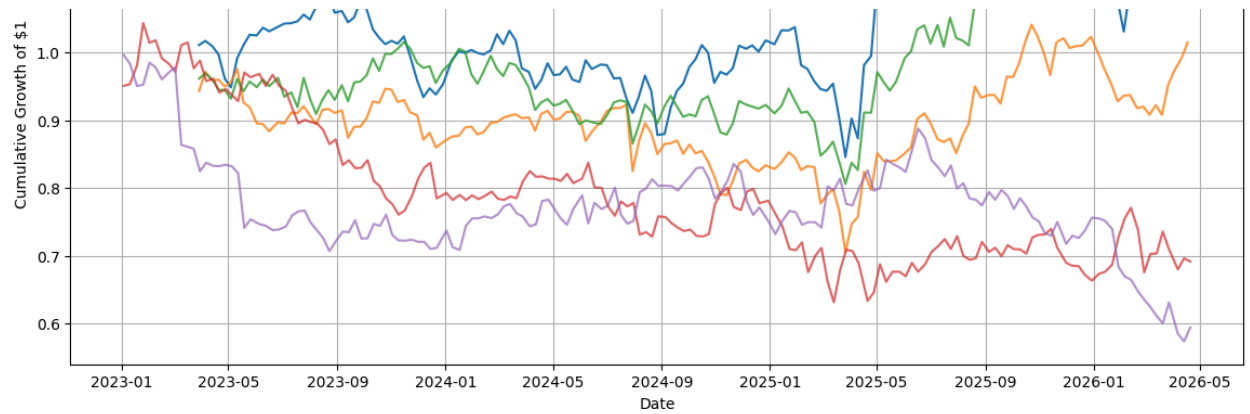
dtype: float32

Top Tree Permutation Features:

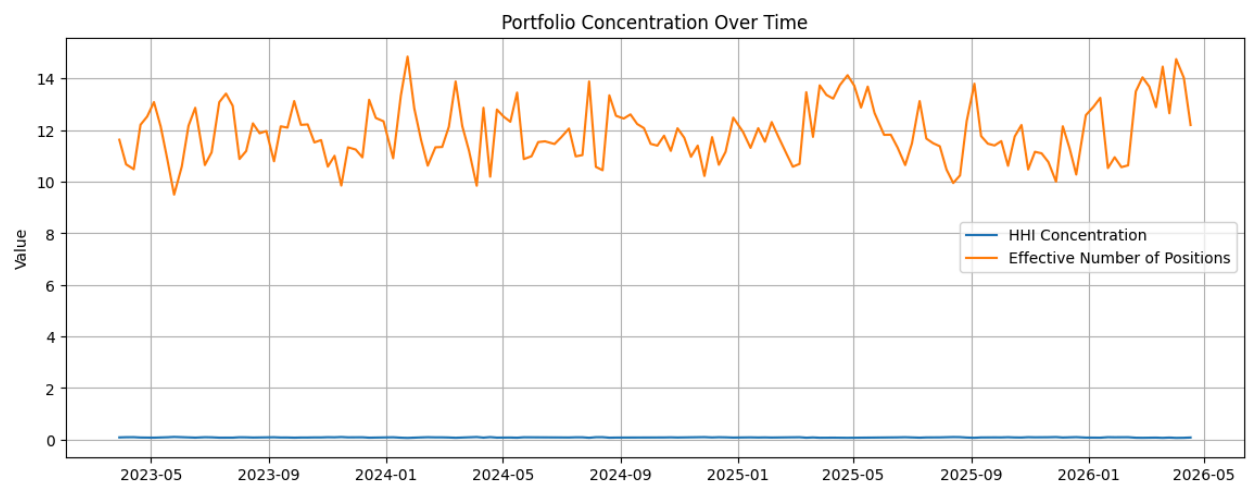
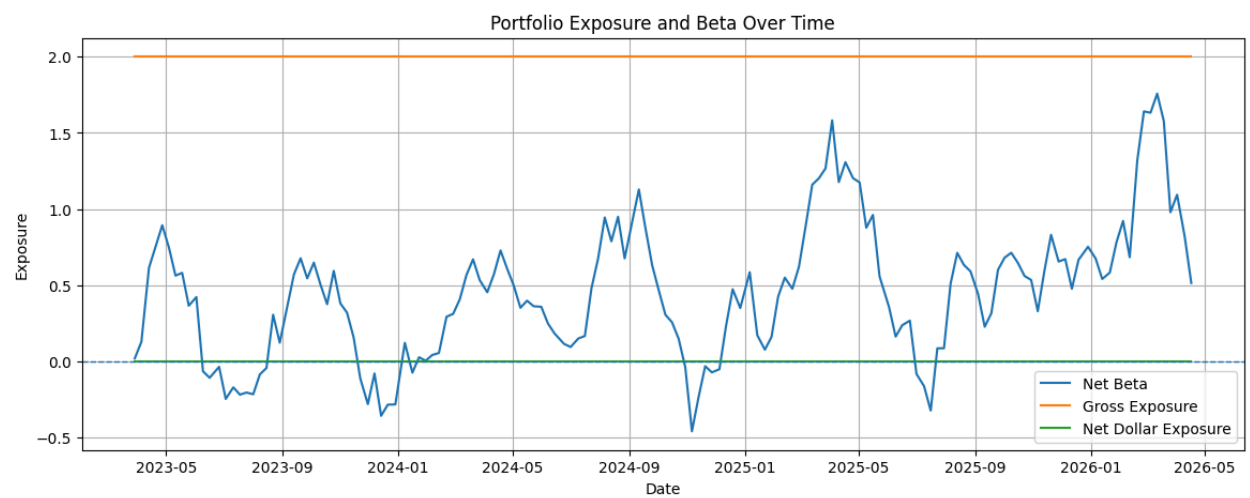
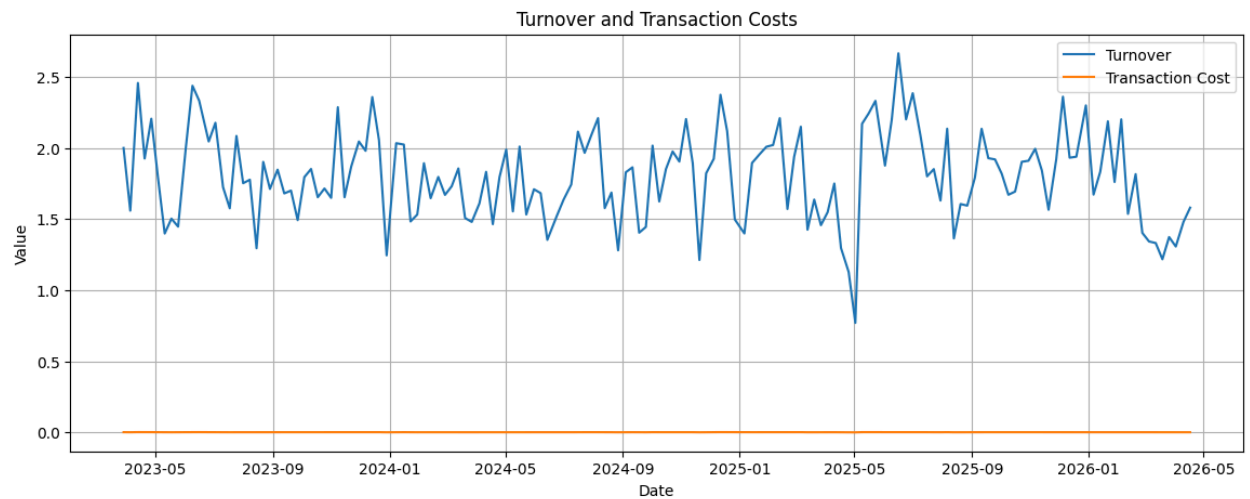
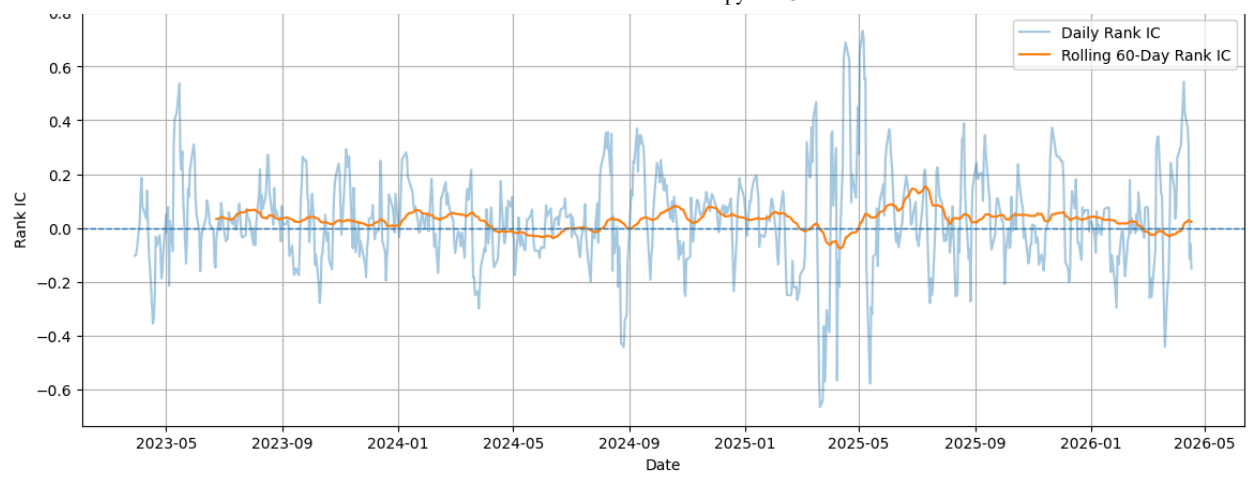
vol_21d_rank	0.015743
vol_21d_cs_z	0.011648
ret_63d_rank	0.004634
price_to_sma_63_cs_z	0.004402
vol_63d	0.002595
ret_10d	0.001267
excess_ret_5d	0.000902
excess_ret_1d_cs_z	0.000858
price_to_sma_21_cs_z	0.000771
volume_chg_1d	0.000296
ret_1d	0.000227
close_open_ret	0.000134
dollar_volume_to_sma_21	0.000100
daily_range	0.000050
vol_5d	-0.000118
price_to_sma_10_cs_z	-0.000137
close_position	-0.000167
ret_5d_rank	-0.000316
volume_to_sma_21	-0.000415
ret_21d_rank	-0.000417
ret_63d	-0.000670
ret_21d	-0.000676
vol_21d	-0.001285
ret_5d	-0.001788

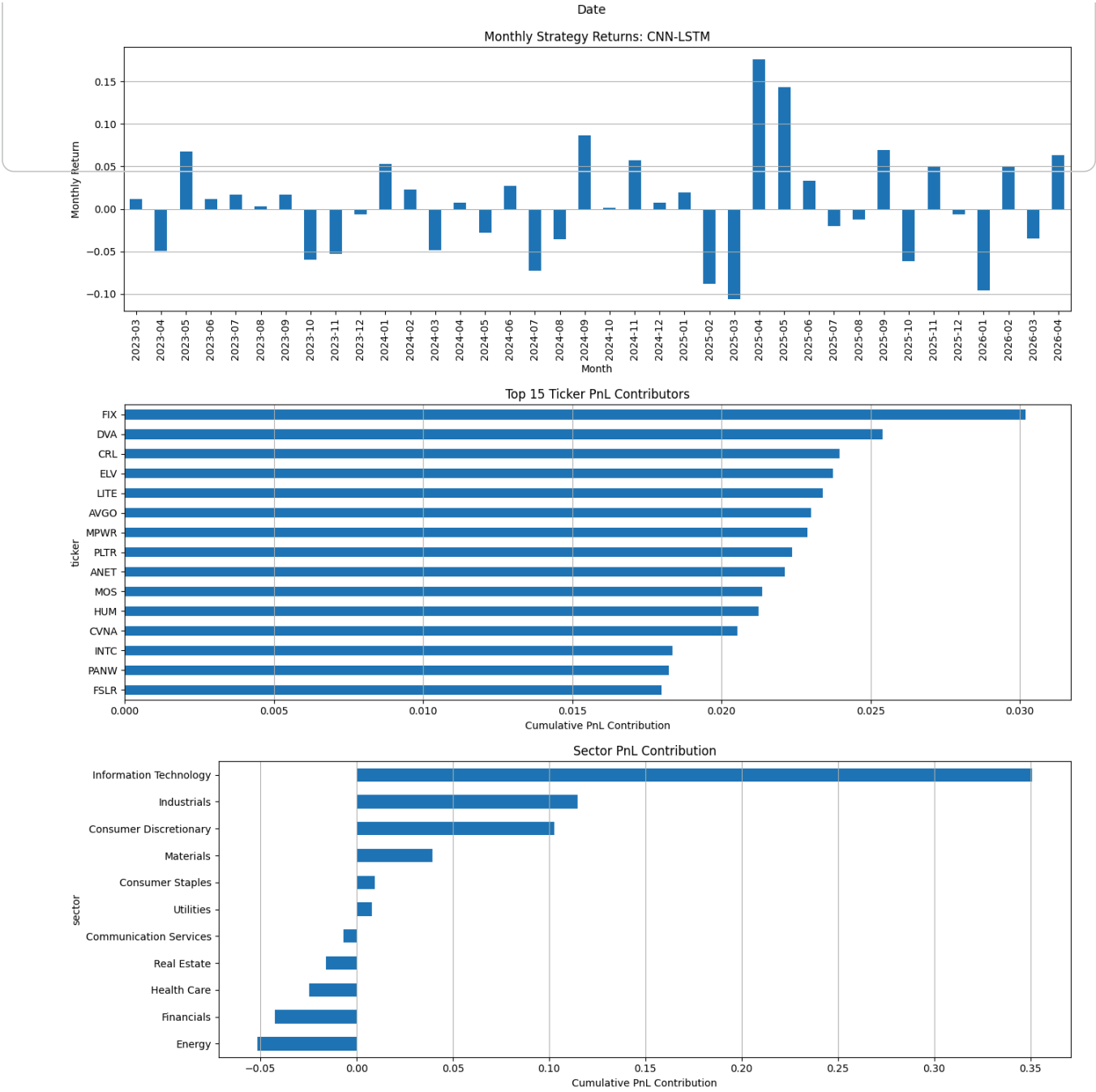
dtype: float64





o o CNN-LSTM Rank IC Over Time





=====

FINAL INTERPRETATION

=====

This version keeps the original 24 model input features but upgrades the research pipelir

```

meta_test_lstm[["date", "ticker", "pred_ret_5d", "realized_ret_5d"]].to_csv(
    "lstm_preds_sp500.csv", index=False
)
from google.colab import files
files.download("lstm_preds_sp500.csv")

```

```

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats

PERIODS_PER_YEAR = 252 / REBALANCE_EVERY
RF_ANNUAL          = 0.0
STRATEGY           = "CNN-LSTM-Attn"
PLOT_DIR           = "eval_plots"
os.makedirs(PLOT_DIR, exist_ok=True)

print(f"REBALANCE_EVERY = {REBALANCE_EVERY} -> PERIODS_PER_YEAR = {PERIODS_PER_YEAR:.2f}

```

```

REBALANCE_EVERY = 5 -> PERIODS_PER_YEAR = 50.40

```

```

STRATEGY_BT = {
    "CNN-LSTM-Attn": bt_lstm,
    "Ridge":         bt_ridge,
    "Tree/XGBoost":  bt_tree,
    "Momentum":      bt_mom,
    "Mean Reversion": bt_mr,
}
STRATEGY_HOLDINGS = {
    "CNN-LSTM-Attn": holdings_lstm,
    "Ridge":         holdings_ridge,
    "Tree/XGBoost":  holdings_tree,
    "Momentum":      holdings_mom,
    "Mean Reversion": holdings_mr,
}

bt_df          = STRATEGY_BT[STRATEGY].copy()
holdings_df    = STRATEGY_HOLDINGS[STRATEGY].copy()

# Ensure datetime index on returns
bt_df["date"]   = pd.to_datetime(bt_df["date"])
holdings_df["date"] = pd.to_datetime(holdings_df["date"])

# r = portfolio return series at 5-day rebalance frequency
r = bt_df.set_index("date")["net_return"].astype(float).dropna()

# w = wide weights (date x ticker), built from long-format holdings
w = (holdings_df.pivot_table(index="date", columns="ticker",
                             values="weight", fill_value=0.0)
     .reindex(r.index, fill_value=0.0))

# REBAL = list of rebalance dates (every row in bt_df IS a rebalance)
REBAL = pd.DatetimeIndex(r.index)

print(f"{STRATEGY}: {len(r)} rebalance observations "
      f"({r.index.min().date()} -> {r.index.max().date()})")

```

```
print(f"Universe touched: {w.shape[1]} unique tickers")
```

CNN-LSTM-Attn: 153 rebalance observations (2023-03-29 -> 2026-04-10)
Universe touched: 495 unique tickers

```
spy_df = spy.copy()
spy_df["date"] = pd.to_datetime(spy_df["date"])
spy_daily = spy_df.set_index("date")["spy_ret_1d"].astype(float).dropna()

# Compound daily SPY returns into the same 5-day buckets the strategy uses.
# For each rebalance date r.index[i], take the daily SPY returns from
# r.index[i-1]+1 day through r.index[i].
bench_buckets = []
prev = None
for d in r.index:
    if prev is None:
        bench_buckets.append(np.nan)
    else:
        chunk = spy_daily.loc[(spy_daily.index > prev) & (spy_daily.index <= d)]
        bench_buckets.append((1 + chunk).prod() - 1 if len(chunk) else 0.0)
    prev = d

b = pd.Series(bench_buckets, index=r.index, name="benchmark").fillna(0.0)
print(f"Benchmark aligned: {b.notna().sum()} bucketed observations")

try:
    TICKER_PNL = ticker_pnl
except NameError:
    TICKER_PNL = holdings_df.groupby("ticker")["pnl_contribution"].sum().sort_values(ascending=True)
```

Benchmark aligned: 153 bucketed observations

```
def period_metrics(r: pd.Series, periods_per_year: float = PERIODS_PER_YEAR) -> pd.Series:
    eq = (1 + r).cumprod()
    n = len(r)
    years = n / periods_per_year

    total_ret = eq.iloc[-1] - 1.0
    ann_ret = (1 + total_ret) ** (1 / years) - 1
    ann_vol = r.std() * np.sqrt(periods_per_year)
    rf_p = RF_ANNUAL / periods_per_year
    sharpe = (r.mean() - rf_p) / r.std() * np.sqrt(periods_per_year)

    downside = r[r < 0]
    dd_dev = downside.std() * np.sqrt(periods_per_year) if len(downside) else np.nan
    sortino = ((r.mean() - rf_p) / downside.std() * np.sqrt(periods_per_year)
               if len(downside) else np.nan)

    running = eq.cummax()
    dd_series = eq / running - 1.0
    max_dd = dd_series.min()
    calmar = ann_ret / abs(max_dd) if max_dd < 0 else np.nan

    skew = stats.skew(r, bias=False)
    kurt = stats.kurtosis(r, fisher=True, bias=False)
    ulcer = np.sqrt((dd_series ** 2).mean())

    trough = dd_series.idxmin()
    peak = eq.loc[:trough].idxmax()
```

```

after = eq.loc[trough:]
rec_pts = after[after >= eq.loc[peak]]
recovery_periods = (rec_pts.index[0] - trough).days if len(rec_pts) else np.nan

return pd.Series({
    "Total return": total_ret,
    "Annualized return": ann_ret,
    "Annualized volatility": ann_vol,
    "Sharpe ratio": sharpe,
    "Sortino ratio": sortino,
    "Max drawdown": max_dd,
    "Calmar ratio": calmar,
    "Skewness (per-period)": skew,
    "Excess kurtosis": kurt,
    "Downside deviation (ann.)": dd_dev,
    "Ulcer index": ulcer,
    "Recovery (calendar days)": recovery_periods,
})

period_tbl = period_metrics(r)
period_tbl

```

	0
Total return	0.400204
Annualized return	0.117268
Annualized volatility	0.193251
Sharpe ratio	0.668868
Sortino ratio	1.165379
Max drawdown	-0.186609
Calmar ratio	0.628414
Skewness (per-period)	0.892885
Excess kurtosis	4.388670
Downside deviation (ann.)	0.110916
Ulcer index	0.059444
Recovery (calendar days)	248.000000

dtype: float64

```

def trade_metrics(holdings_long: pd.DataFrame,
                  weights: pd.DataFrame) -> tuple[pd.Series, pd.DataFrame]:
    """
    holdings_long: long format with columns date, ticker, side, weight, pnl_contribution
    weights:       wide  date x ticker
    """

    h = holdings_long.copy()
    h["sign"] = np.where(h["side"].str.lower().str.startswith("l"), 1, -1)

    trades = []
    for ticker, grp in h.sort_values(["ticker", "date"]).groupby("ticker"):
        grp = grp.reset_index(drop=True)
        # Detect breaks in continuous holding (nan > 1 rebalance period

```

```

# or sign change opens a new trade)
new_trade = (grp["date"].diff().dt.days.gt(REBALANCE_EVERY * 2)
              | grp["sign"].diff().ne(0))
new_trade.iloc[0] = True
trade_id = new_trade.cumsum()
for tid, block in grp.groupby(trade_id):
    trades.append({
        "ticker":      ticker,
        "side":         "Long" if block["sign"].iloc[0] > 0 else "Short",
        "entry":         block["date"].iloc[0],
        "exit":          block["date"].iloc[-1],
        "n_periods":     len(block),
        "holding_days":  (block["date"].iloc[-1] - block["date"].iloc[0]).days
                        + REBALANCE_EVERY,
        "avg_weight":    block["weight"].abs().mean(),
        "pnl":           block["pnl_contribution"].sum(),
    })
trades_df = pd.DataFrame(trades)

n = len(trades_df)
wn = trades_df[trades_df["pnl"] > 0]
ls = trades_df[trades_df["pnl"] < 0]
win_rate = len(wn) / n if n else np.nan
avg_w_ = wn["pnl"].mean() if len(wn) else np.nan
avg_l_ = ls["pnl"].mean() if len(ls) else np.nan
pf = wn["pnl"].sum() / abs(ls["pnl"].sum()) if len(ls) and ls["pnl"].sum() != 0 else np.nan
avg_hold = trades_df["holding_days"].mean() if n else np.nan

# Turnover per rebalance: |Δw| summed across the cross-section, averaged over time
turnover_per_rebal = weights.diff().abs().sum(axis=1).iloc[1:].mean()

out = pd.Series({
    "Number of trades":      n,
    "Win rate (closed trades)": win_rate,
    "Average win":           avg_w_,
    "Average loss":          avg_l_,
    "Avg win / |Avg loss|":  (avg_w_ / abs(avg_l_)) if avg_l_ else np.nan,
    "Profit factor":         pf,
    "Average holding period (days)": avg_hold,
    "Avg turnover per rebal": turnover_per_rebal,
})
return out, trades_df

trade_tbl, trades_df = trade_metrics(holdings_df, w)
trade_tbl

```

	0
Number of trades	3478.000000
Win rate (closed trades)	0.548879
Average win	0.001628
Average loss	-0.001559
Avg win / Avg loss	1.044794
Profit factor	1.271199
Average holding period (days)	29.785509
Avg turnover per rebalance	1.768225

dtype: float64

```
def risk_metrics(r: pd.Series, w: pd.DataFrame, b: pd.Series,
                bt_df: pd.DataFrame,
                periods_per_year: float = PERIODS_PER_YEAR) -> pd.Series:
    # Exposures
    bt_indexed = bt_df.set_index("date")
    gross = bt_indexed["gross_exposure"] if "gross_exposure" in bt_indexed.columns \
        else w.abs().sum(axis=1)
    net = bt_indexed["net_exposure"] if "net_exposure" in bt_indexed.columns \
        else w.sum(axis=1)

    abs_w = w.abs()
    row_sum = abs_w.sum(axis=1).replace(0, np.nan)
    hhi = (abs_w.div(row_sum, axis=0) ** 2).sum(axis=1).mean()
    top5 = abs_w.div(row_sum, axis=0).apply(
        lambda x: x.nlargest(5).sum(), axis=1).mean()

    # Rolling stats
    if "rolling_vol" in bt_indexed.columns:
        roll_vol_mean = bt_indexed["rolling_vol"].mean()
    else:
        roll_vol_mean = (r.rolling(21).std() * np.sqrt(periods_per_year)).mean()
    if "rolling_sharpe" in bt_indexed.columns:
        roll_sh_mean = bt_indexed["rolling_sharpe"].mean()
    else:
        roll_sh_mean = (r.rolling(21).mean() / r.rolling(21).std()
                        * np.sqrt(periods_per_year)).mean()

    # Beta / alpha / TE / IR vs benchmark
    aligned = pd.concat([r, b], axis=1).dropna()
    if len(aligned) > 2 and aligned.iloc[:,1].std() > 0:
        cov = np.cov(aligned.iloc[:,0], aligned.iloc[:,1], ddof=1)
        beta = cov[0, 1] / cov[1, 1]
        alpha_ann = (aligned.iloc[:,0].mean() - beta * aligned.iloc[:,1].mean()) * periods_per_year
        active = aligned.iloc[:,0] - aligned.iloc[:,1]
        te = active.std() * np.sqrt(periods_per_year)
        ir = (active.mean() / active.std() * np.sqrt(periods_per_year)
              if active.std() > 0 else np.nan)
    else:
        beta = alpha_ann = te = ir = np.nan

    var95 = np.quantile(r, 0.05)
    cvar95 = r[r <= var95].mean()
    var99 = np.quantile(r, 0.01)
```